

Module: Problem 1

This module contains two definitions of a function named `f`. Both implementations demonstrate recursive patterns that lack proper termination conditions, which can lead to infinite recursion and eventual stack overflow. The first version attempts to recurse while `x` is greater than 3, and the second version builds a list while recursing for all positive values of `x`. Neither function returns a value; they implicitly return `None`.

Functions

`f(x)` – first definition

```
def f(x):  
    while x > 3:  
        f(x+1)
```

Description:

Recursively calls itself with an incremented argument (`x + 1`) as long as the original `x` is greater than 3. Because the loop condition never modifies `x`, the recursion never reaches a terminating condition, resulting in infinite recursion.

Parameters: - `x` (*int*): The starting integer value that determines whether the recursion will be entered. The function expects a numeric type that supports comparison with `3` and addition.

Returns:

`None` – the function does not return a value explicitly.

Examples:

```
# No runtime captures are available; the function would recurse indefinitely.  
# The following call will eventually raise a RecursionError.  
# f(5)  # → RecursionError: maximum recursion depth exceeded
```

Edge Cases / Notes: - The loop condition `while x > 3` never updates `x`, so the body executes forever once entered. - Calling `f` with any integer `x > 3` will cause infinite recursion and typically terminate with a `RecursionError`. - Calling `f` with `x <= 3` results in the function exiting immediately, returning `None`.

`f(x)` – second definition

```
def f(x):  
    a = []  
    while x > 0:  
        a.append(x)  
        f(x-1)
```

Description:

Creates a local list `a` and, while `x` is positive, appends the current value of `x` to the list and recursively calls

itself with `x-1` . Like the first version, it lacks a proper termination condition for the recursive call inside the loop, leading to potentially unbounded recursion.

Parameters: - `x` (*int*): The starting integer value; the loop continues while `x` is greater than 0.

Returns:

`None` – the function does not return the constructed list; it implicitly returns `None` .

Examples:

```
# No runtime captures are available; the function will recurse until the recursion limit.  
# f(3)  # → RecursionError: maximum recursion depth exceeded
```

Edge Cases / Notes: - Each recursive call creates its own list `a` ; the lists are never returned or used outside their respective call frames. - For any `x > 0` , the recursion depth grows roughly linearly with the initial `x` , quickly exhausting the Python recursion limit. - If `x <= 0` , the `while` loop body never executes and the function returns immediately (`None`).